

AI Project: Titanic

TA Name: Esraa Ali

Team Members:

Name	ID
ساره حسن عبد المغني محمد	20241700361
سماء محمد علي عمر	20241700395
ساره جمال حسنى حميده	20241700359
سلمي محمود عبد القادر القصبي	20241700386
سارة عصام ابراهيم عبيد	20241700364
سلسبيل عادل احمد	20241700376

1. Introduction

The Titanic disaster is one of the most well-known accidents in history. During the sinking of the Titanic, not all passengers survived due to several factors such as age, gender, passenger class, and ticket fare. Therefore, predicting survival based on passenger information became a popular machine learning classification problem.

The main objective of this project is to build a machine learning model capable of predicting whether a passenger survived or not using the Titanic dataset. The dataset contains various features describing passengers,

Several preprocessing techniques, data analysis methods, visualization techniques, and machine learning models were applied in order to improve prediction performance and achieve accurate classification results.

2. Data Preprocessing & Feature Engineering

Before training the machine learning models, several preprocessing and feature engineering techniques were applied to improve data quality, enhance feature representation, and increase model performance.

Dataset Exploration

The dataset was initially explored using functions such as `head()`, `info()`, and `describe()` to understand the structure of the data, feature types, statistical summaries, and missing values.

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S

The dataset was also checked for duplicate rows using the `duplicated()` function. No significant duplicate records were found.

Stratified Data Splitting

The dataset was divided into training and validation sets using `StratifiedShuffleSplit` in order to preserve the original distribution of the target variable (Survived) in both sets.

This technique helped prevent biased training and evaluation results.

Avoiding Data Leakage

Copies of the training, validation, and test datasets were created before preprocessing to avoid unintentional modifications and data leakage during experimentation.

In addition, all preprocessing operations that required statistical calculations were performed using only the training dataset. For example:

Missing Age values in the validation and test sets were filled using median values calculated from the training set.

Missing Embarked values were replaced using the mode from the training set.

Missing Fare values in the test set were filled using the training set median.

Feature scaling was fitted only on the training dataset, then applied to the validation and test datasets.

This approach ensured that no information from the validation or test datasets leaked into the training process, resulting in more reliable and unbiased model evaluation.

Feature Engineering

Several new features were created to improve model performance and extract hidden information from the dataset.

FamilySize Feature

A new feature called FamilySize was created by combining the SibSp and Parch features. This feature represents the total number of family members traveling together.

IsAlone Feature

Another binary feature called IsAlone was created to determine whether a passenger traveled alone or with family members.

Passengers traveling alone showed lower survival rates compared to passengers traveling with relatives.

Title Extraction

Passenger titles such as Mr, Mrs, Miss, and Dr were extracted from passenger names using regular expressions.

Rare titles were grouped into a common category called Other in order to reduce feature cardinality and improve model generalization.

Handling Missing Values

Several missing value handling techniques were applied:

Missing Age values were filled using the median age grouped by passenger title. This method provided more meaningful estimations than using the global median.

Missing Embarked values were replaced using the mode value from the training dataset.

Missing Fare values in the test set were filled using the median fare value from the training set.

Encoding Categorical Features

Categorical variables were converted into numerical representations suitable for machine learning models.

One-Hot Encoding was applied on Embarked and Title features using `pd.get_dummies()`.

The Sex feature was converted into binary values:

male = 0

female = 1

Feature Dropping

Some features were removed because they were either irrelevant, highly incomplete, or contained non-useful information for prediction.

The removed features were:

Name

Ticket

Cabin

PassengerId

Cabin was removed because it contained a large number of missing values, while Ticket and Name were considered less useful after extracting passenger titles.

Ensuring Consistent Feature Structure

After applying One-Hot Encoding, some categorical values appeared in one dataset but not in others. This caused differences in the number and order of features between the training, validation, and test datasets.

To solve this issue:

The validation dataset was reindexed using all training dataset columns.

The test dataset was reindexed using the training dataset columns excluding the target variable (Survived).

Missing columns were filled with zeros using fill_value=0.

This step ensured that:

All datasets had identical feature structures.

The models received the same input format during training and prediction.

Feature mismatch errors were prevented during evaluation and testing.

Outlier Analysis

Boxplots were generated to detect outliers in numerical features such as:

Fare

Age

SibSp

Parch

FamilySize

Initially, outlier handling using clipping based on the IQR method was tested on features such as Fare and Age. However, experimental results showed that removing aggressive outlier handling produced slightly better model performance.

This indicates that extreme values, especially Fare values, carried important information related to passenger socioeconomic status and survival probability.

Therefore, the final models were trained without aggressive outlier clipping.

Feature Scaling

Feature scaling was applied using StandardScaler on the following numerical features:

Age

Fare

SibSp

Parch

FamilySize

StandardScaler transforms features so that they have:

Mean = 0

Standard Deviation = 1

Feature scaling was especially important for machine learning algorithms such as:

- Support Vector Machine (SVM)
- Logistic Regression

because these models are sensitive to differences in feature magnitudes.

To avoid data leakage:

The scaler was fitted only on the training dataset using `fit_transform()`.

The same scaler was then applied to the validation and test datasets using `transform()` only.

This ensured that no information from the validation or test datasets influenced the training process.

Feature Selection Experiments

Several experiments were performed to evaluate the impact of different features on model performance.

The SibSp and Parch features were temporarily removed because FamilySize and IsAlone were already derived from them. However, removing these features slightly reduced model accuracy.

This suggests that SibSp and Parch still contained additional useful information beyond the engineered features.

Therefore, the final models retained:

SibSp

Parch

FamilySize

IsAlone

in order to achieve better predictive performance.

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	FamilySize	IsAlone	Embarked_C	Embarked_Q	Embarked_S	Title_Master	Title_Miss	Title_Mr	Title_Mrs	Title_Other
692	1	3	0	0.033471	-0.465084	-0.466183	0.513812	-0.556339	1	False	False	True	False	False	True	False	False
481	0	2	0	0.033471	-0.465084	-0.466183	-0.662563	-0.556339	1	False	False	True	False	False	True	False	False
527	0	1	0	0.033471	-0.465084	-0.466183	3.955399	-0.556339	1	False	False	True	False	False	True	False	False
855	1	3	1	-0.872714	-0.465084	0.727782	-0.467874	0.073412	0	False	False	True	False	False	False	True	False
801	1	2	1	0.108986	0.478335	0.727782	-0.115977	0.703162	0	False	False	True	False	False	False	True	False

3. Models

Decision Tree Classifier

Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks. In this project, it was used to predict passenger survival in the Titanic dataset by learning decision rules inferred from the features.

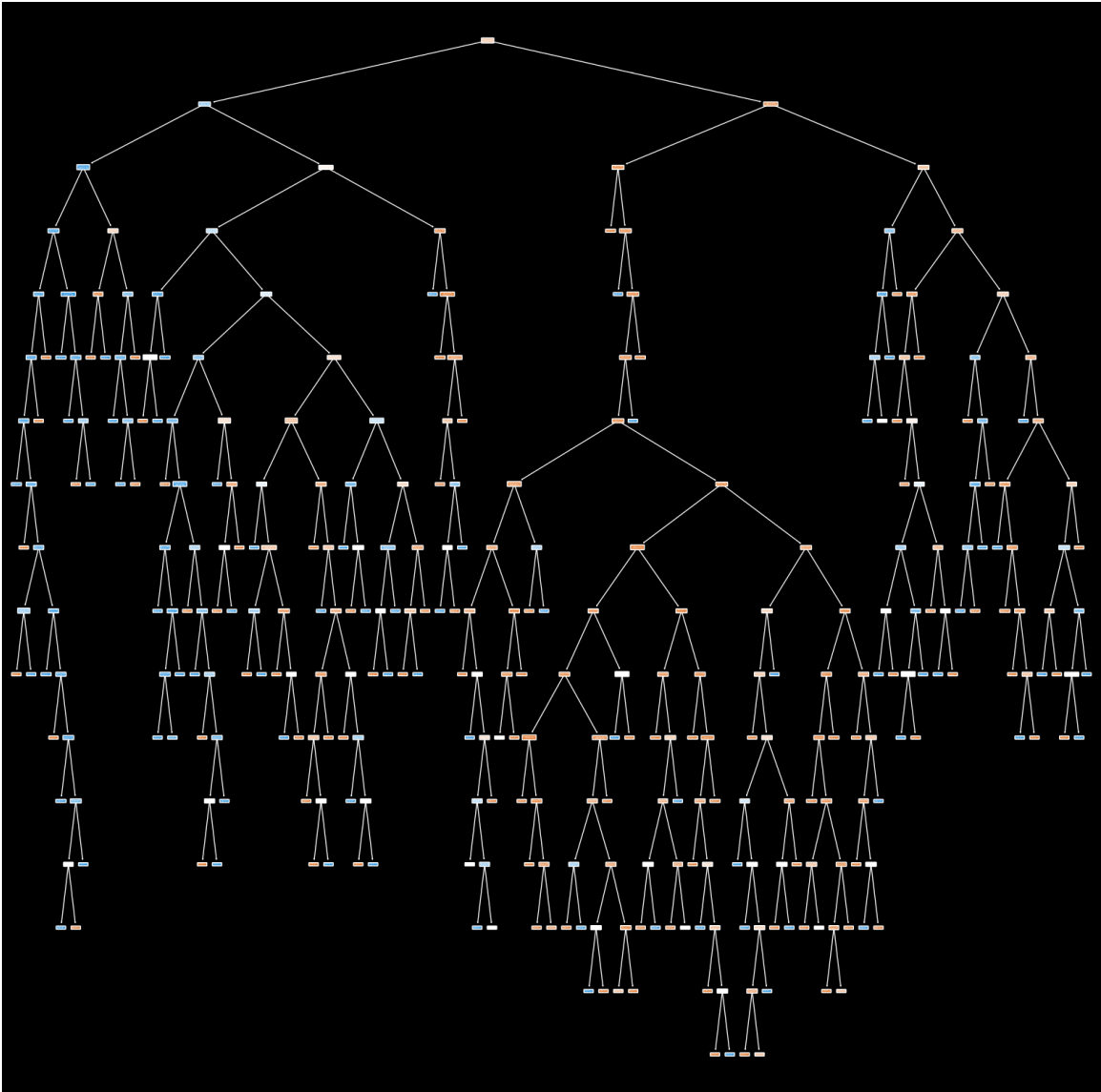
Initial Model Training

A baseline Decision Tree model was first trained using default parameters without any hyperparameter tuning. The model was fitted on the training dataset and evaluated on the validation set.

The performance was measured using:

Accuracy Score ,Classification Report

Additionally, the structure of the decision tree was visualized to understand how the model splits the data and makes predictions.



Hyperparameter Tuning

GridSearchCV was applied to improve model performance and reduce overfitting by finding the optimal set of hyperparameters.

The following parameters were tuned:

`max_depth={3, 5, 7}`

`min_samples_split={2, 3, 4}`

`min_samples_leaf={1, 2, 3}`

criterion={ entropy}

Parameter Explanation:

max_depth controls the maximum depth of the tree and helps prevent overfitting.

min_samples_split defines the minimum number of samples required to split a node.

min_samples_leaf controls the minimum samples required in a leaf node.

criterion (entropy) measures the quality of a split based on information gain.

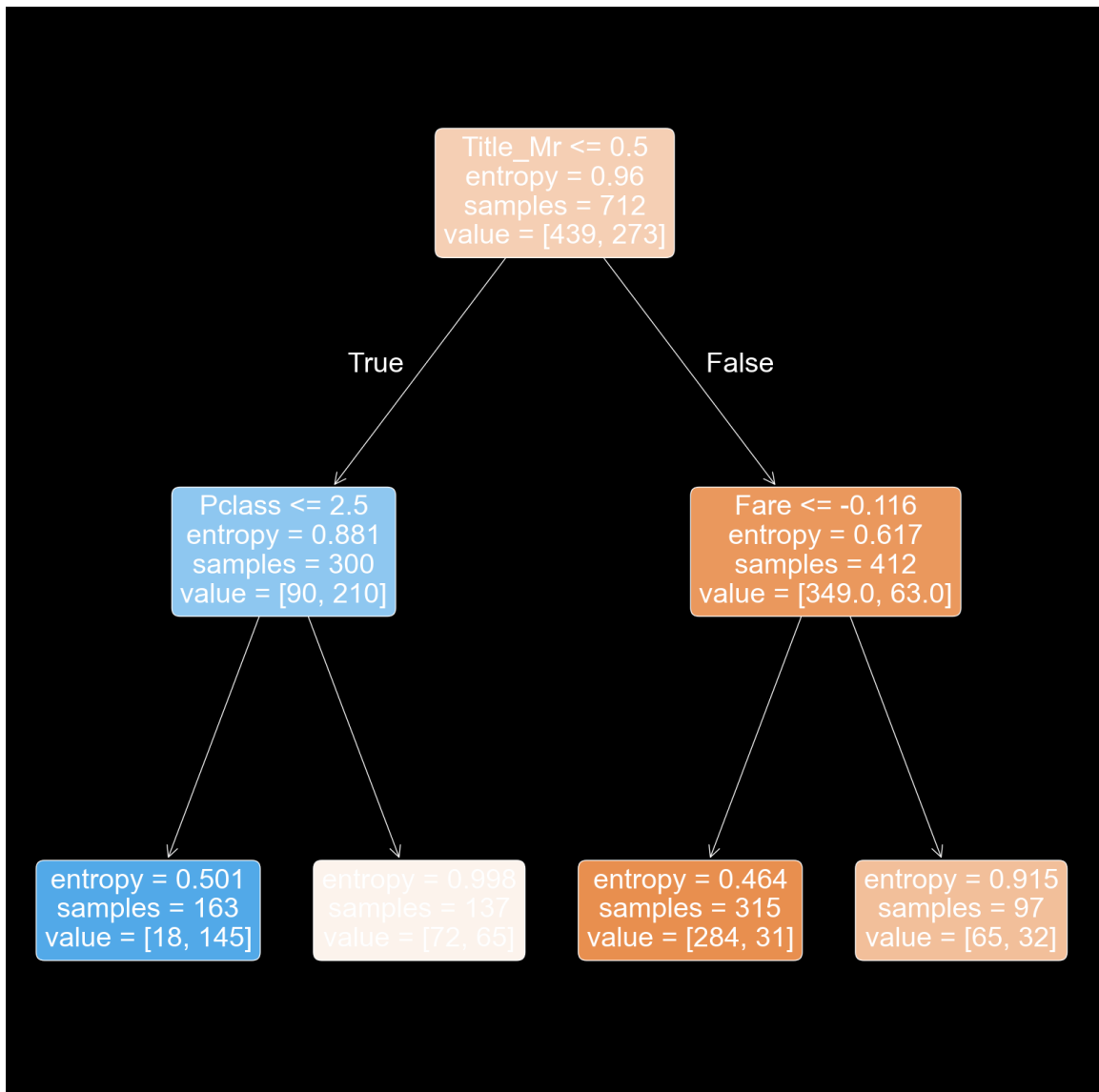
Cross-validation (cv = 5) was used to ensure reliable model evaluation during hyperparameter tuning.

Best Model Selection

The best combination of hyperparameters was selected automatically by GridSearchCV based on validation accuracy. The optimized model was then used to make predictions on the validation dataset.

A second decision tree was also manually tested with specific hyperparameters to compare performance, which resulted in lower accuracy compared to the tuned

model. This confirms the effectiveness of hyperparameter tuning.



Model Evaluation

After training the initial Decision Tree model with default parameters, the evaluation results were as follows:

Accuracy: 83.24 %

The classification report provided detailed performance analysis for both survival classes

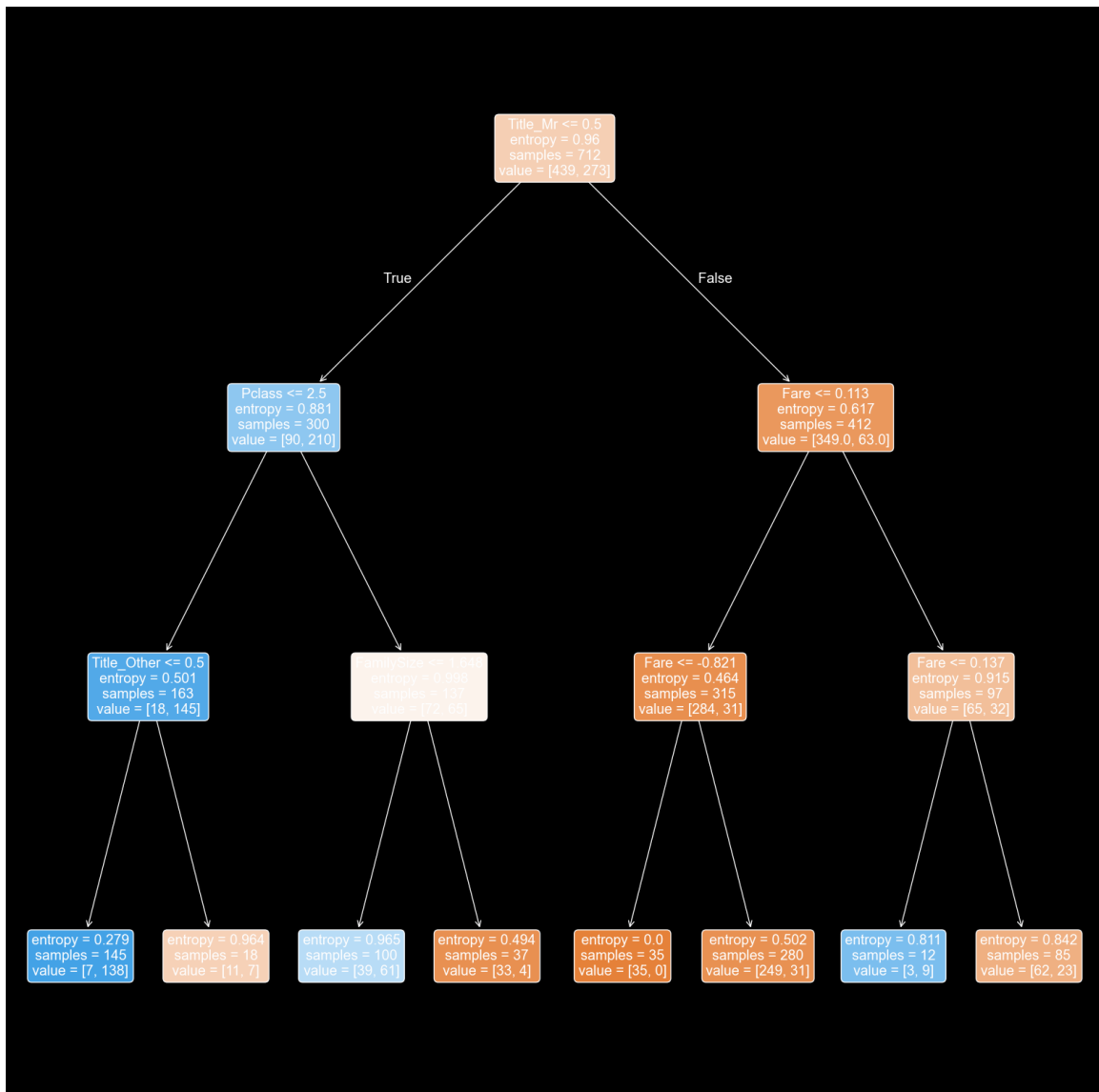
	precision	recall	f1-score	support
0	0.85	0.88	0.87	110
1	0.80	0.75	0.78	69

accuracy 0.83 179

macro avg 0.83 0.82 0.82 179

weighted avg 0.83 0.83 0.83 179

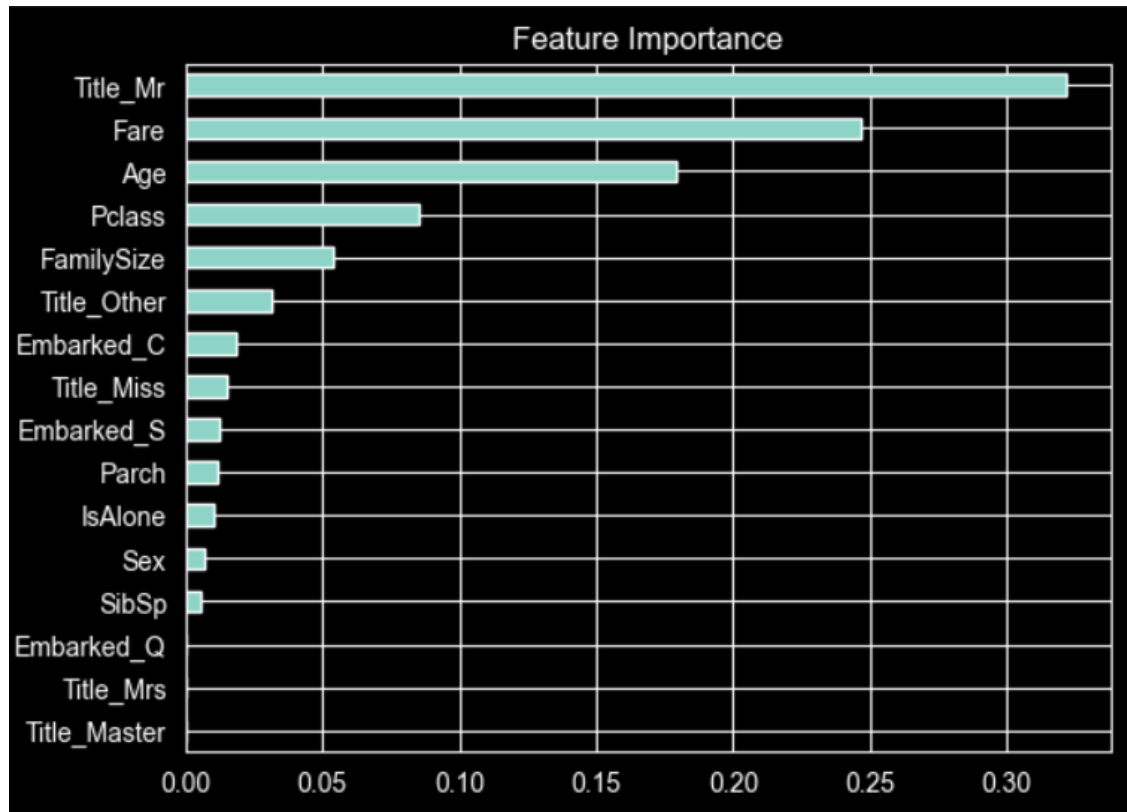
The final trained decision tree was visualized to interpret the learned decision rules. This helps in understanding how different features contribute to survival prediction.



However, deeper trees may lead to overfitting, which is why controlling depth through hyperparameters was important.

Feature Importance

Feature importance was extracted from the trained model to identify which features had the most influence on predictions.



Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification task.

In this project, SVM was used to predict passenger survival in the Titanic dataset.

Data Preparation

Before training the model:

Missing values were handled

Categorical data was encoded

The dataset was split into training and validation sets

Features were scaled using StandardScaler

Model Implementation

The model was implemented using Scikit-learn with:

SVC (Support Vector Classifier)

Pipeline

GridSearchCV for hyperparameter tuning

A machine learning pipeline was created to combine:

Feature scaling using StandardScaler

SVM classification

Using a pipeline ensured that preprocessing and model training were applied consistently during cross validation and prediction.

Hyperparameter Tuning

GridSearchCV was used to search for the best hyperparameter combination for the SVM model.

The following parameters were tested:

$C = \{1, 10\}$

Kernel = { linear, rbf }

Gamma = { scale }

Parameter Description

C controls the trade-off between maximizing the margin and minimizing classification errors.

Kernel determines the transformation applied to the data:

linear kernel or radial basis function (rbf) kernel

Gamma controls the influence of individual training samples in the RBF kernel.

Cross Validation

Cross validation with 3 folds (cv = 3) was used during Grid Search to evaluate multiple parameter combinations and reduce overfitting risk.

The best-performing model was automatically selected based on validation accuracy.

Model Evaluation

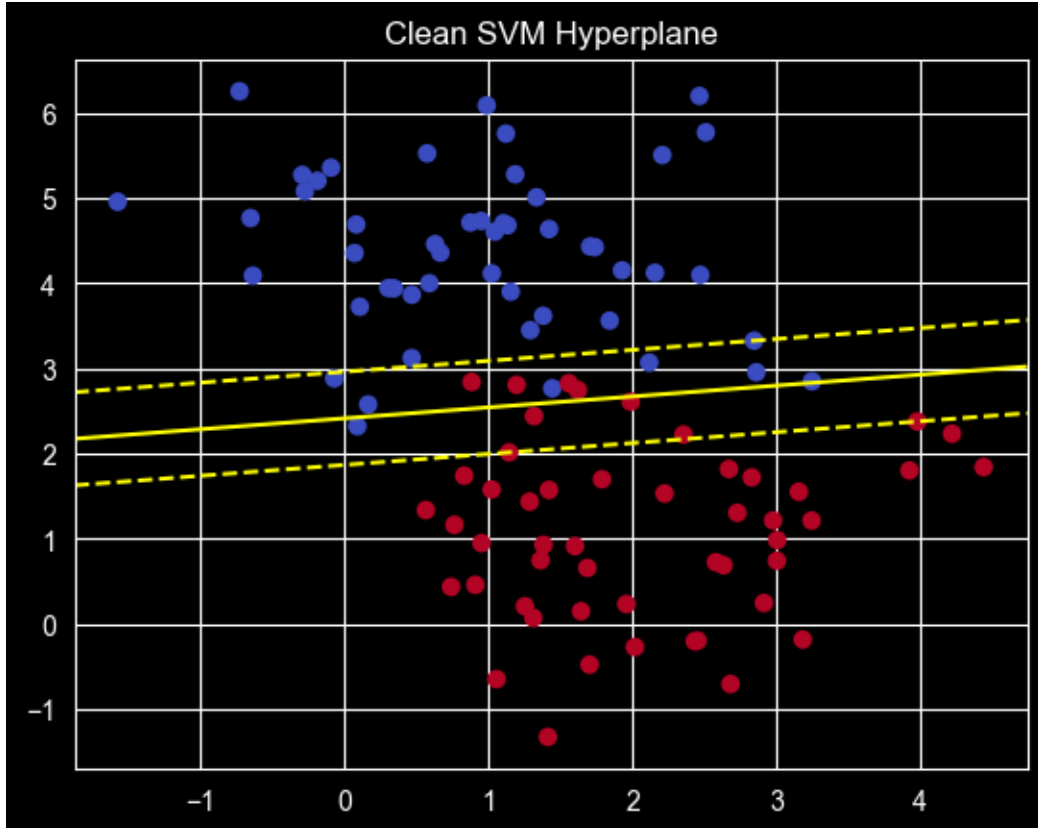
After hyperparameter tuning, the best SVM model was evaluated using the validation dataset.

The classification report provided detailed performance analysis for both survival classes

	precision	recall	f1-score	support
0	0.83	0.90	0.86	110
1	0.82	0.71	0.76	69
accuracy			0.83	179
macro avg	0.82	0.81	0.81	179
weighted avg	0.83	0.83	0.82	179

Accuracy: 82.6816 %

A confusion matrix was also generated to visualize correct and incorrect predictions made by the model.



Logistic Regression Model

Logistic Regression was used as one of the baseline classification models for predicting .This algorithm is widely used for binary classification problems .

Initial Model Evaluation

Before performing hyperparameter tuning, a Logistic Regression model was trained using the default parameters provided by Scikit-learn. The model was then evaluated on the validation dataset in order to establish a baseline performance.

The following steps were performed:

The target variable (Survived) was separated from the input features.

The model was trained using the training dataset.

Predictions were generated on the validation dataset.

Accuracy score was calculated to evaluate the initial model performance.

This initial evaluation helped measure the improvement achieved later through hyperparameter tuning.

Hyperparameter Tuning

GridSearchCV was applied to search for the best combination of hyperparameters and improve model performance.

The following hyperparameters were tested:

$C = \{0.01, 0.1, 1, 10, 100\}$

$\text{Penalty} = \{l1, l2\}$

$\text{Solver} = \{\text{liblinear}, \text{saga}\}$

Parameter Description

The parameter C controls the regularization strength:

Smaller values increase regularization.

Larger values reduce regularization.

L1 Regularization /L2 Regularization :were tested in order to identify the best generalization performance.

Cross Validation

Stratified K-Fold Cross Validation was used during hyperparameter tuning with:

Stratification ensured that each fold preserved the same survival class distribution as the original dataset, resulting in more reliable validation performance.

Best Model Selection

After testing multiple parameter combinations, GridSearchCV automatically selected the best-performing Logistic Regression model based on validation accuracy.

Model Evaluation

The optimized Logistic Regression model was evaluated using several classification metrics:

Accuracy Score = 85,47%

LogisticRegression report:

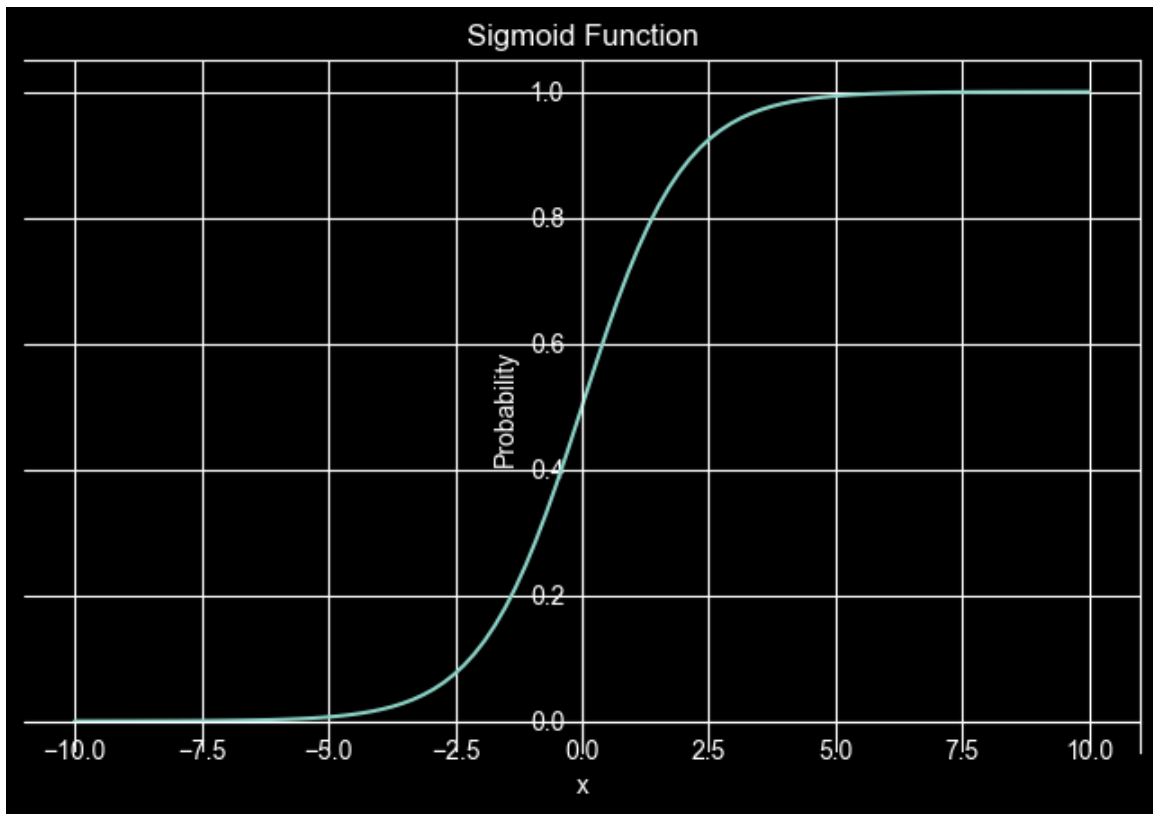
The classification report provided detailed performance analysis for both survival classes.

	precision	recall	f1-score	support
0	0.86	0.91	0.88	110
1	0.84	0.77	0.80	69
accuracy			0.85	179
macro avg	0.85	0.84	0.84	179
weighted avg	0.85	0.85	0.85	179

Accuracy score : 85.47%

A confusion matrix heatmap was also generated to visualize:

Correct predictions -False positives -False negatives



4. Model Evaluation

After training and tuning multiple machine learning models, their performance was evaluated using the validation dataset. The evaluation was based on several classification metrics, including accuracy, precision, recall, and F1-score, in addition to confusion matrix analysis.

1. Performance Comparison

The performance of all trained models is summarized in the following table:

Model	Accuracy
Logistic Regression	85.47%
Decision Tree	83.24 %
Support Vector Machine (SVM)	82.6816 %

2. Best Model Selection

Based on the evaluation results, the best performing model was “Logistic Regression”, as it achieved the highest accuracy along with a better balance between precision and recall compared to the other models.

This indicates that the selected model generalizes better on unseen data and provides more reliable predictions for the survival classification task